Name:- P.Darshith Reg.No:- 192571051 CSA0612

Q77) consider the concept Map:- Algorithm → Nested Loops → operation Growth → Polynomial. complexity → Efficiency. explain how multiple nested loops increase the no. of operations.

Algorithm → Nested loops → operation Growth → Polynomial complexity → Efficiency

- **Algorithm**: step-by-step procedure to solve a Problem
- **Nested loops**: using one Loop inside another loop
- **operation Growth**: No. of operations increases as input size increases
- **Polynomial complexity**: growth can be expressed as a Polynomial function of input size (n)
- **Efficiency**: Determines how well the algorithm scales for large inputs.

WHY NESTED LOOPS INCREASE OPERATIONS?

each additional Loop multiplies the number of interations. If a loop runs n times and another loop runs n times inside it, total operations Become $n \times n = n^2$

EXAMPLES (EXTENDING THE MAP)

$O(n^2)$

Two nested loops

```
For i = 1 to n
For j = 1 to n
    // operation
```

total operation $= n \times n = n^2$

$O(n^3)$

Three nested loops

```
For i = 1 to n
For j = 1 to n
For k = 1 to n
    // operation.
```

Total operations $= n \times n \times n = n^3$.

WHY POLYNOMIAL COMPLEX-ITY AFFECTS SCALABILITY?

- → As n grows, n^2, n^3, --- grow very fast.
- → small increase in input size causes huge increase in operations.
- → Requires more time and resources.
- → Not suitable for very large inputs.

HOW THIS MAP HELPS IDENTITY INEFICIENT ALGORITHM?

- → Highlights the impact of nested loops.
- → helps Predict operation growth early.
- → Identifies algorithm with Identity higher Polynomial orders (e.g., n^3)
- → Guides us to optimize by reducing or eliminating nested loops.

Name :- P-Darshith   Reg.No :- 192571051   CSA0612

Q78) consider the concept map:- Algorithm → Input Reduction → Binary search logarithmic complexity & efficiency. Explain how reducing input size at each steps leads to logarithmic growth.

Algorithm → Input Reduction → Binary search → logarithmic complexity → Efficiency

Algorithm
Step-by-step Procedure to solve a Problem

Input Reduction
Reduce the input size at each step

Binary search
compare middle element and eliminate half of the input each time.

logarithmic complexity
No.of operations grows logarithmically with input size. O(log n)

Efficiency
Requires fewer operation even for large inputs. very efficient.

Sorted Input Requirement [PREREQUISITE]
Binary search works only on sorted data.

HOW INPUT REDUCTION Leads TO LOGARITHMIC GROWTH?

If input size = n
After 1 comparsion → n/2
After 2 comparisons → n/4
After k comparisons → $n/2^k$
stop when $n/2^k = 1$
$\Rightarrow k = \log_2 n$
Total comparisons = O(log n)

WHY BINARY SEARCH IS BETTER THAN LINER SEARCH?

linear search	Binary search.
checks elements one by one	Reduces input by half each step
worst case = O(n)	worst case = O(log n)
more time for large inputs.	much faster for large inputs.

HOW THIS MAP DEMONSTRATES EFFICIENT SEARCHING STRATEGIES?
→ shows the power of reducing problem size
→ logarithmic growth is much slower than linear growth
→ works efficiently for large datasets
→ emphasizes the role of sorted input in achieving efficiency.

EXAMPLE
* Searching in 1 million (10^6 elements)
* Linear search → upto 10^6 comparisons
* Binary search → about $\log_2 10^6$ = 20 comparisions